

Yes — this is **Week 5**, covering:

- **Session: Feature Construction with SQL**
- **Practical: Feature construction exercises in SQL**
- **Session: Working with Temporal Data I**
- **Practical: Building temporal datasets from transactional data**

Below are your **exam-focused Week 5 notes** in the same two formats.

Week 5 — Feature Construction in SQL

Plus Temporal Data

Part A — Structured & Formal Summary

1. Purpose of Week 5

Week 5 is about how to turn raw relational / transactional data into an actual machine learning table.

In machine learning, the model usually needs:

- **X** : input feature table
- **y** : output / target feature

The lecture stresses that creating these tables is a major part of real machine learning work. Since real business data is usually stored in databases, SQL is often the most natural tool for feature construction. The slides describe SQL as useful because it lets us process tabular data where it already lives, instead of loading many large tables into Python and causing memory, transfer and code-complexity problems.

The main Week 5 message is:

Before writing SQL, define what one row represents, what you are predicting, what input features you need, and where each feature comes from.

Session — Feature Construction with SQL

2. Why Use SQL for Feature Construction?

SQL is useful for machine learning preprocessing because:

- business data is often already stored in databases,

- SQL is designed for tabular data,
- many features are aggregations over many rows,
- databases can process large datasets more efficiently than local Python,
- SQL avoids unnecessary data transfer,
- SQL can produce the final machine learning input table directly.

The lecture warns against loading all database tables as separate Python DataFrames when the dataset is large. This can create memory issues, more complex code, more transfer overhead and more work. The desired pattern is: **tables in, table out**.

3. Feature Mining / Feature Engineering Questions

Before implementation, ask:

Question	Meaning
What are we predicting?	Define the output feature and what one row represents.
What input features do we want?	Decide what information may help prediction.
Why do we want them?	Give a rationale, not just a variable name.
What format should features be in?	Numeric, binary, categorical, aggregated, encoded, etc.
Where do we get the information?	Which database tables or external sources contain it?

The lecture emphasises that these questions are linked. You should ideally think first about what you would want without constraints, then map those ideas to the available data.

4. Example Task: Predict Whether a Customer Is a Student

The main SQL lecture uses this example:

Predict whether a customer is a student from purchasing behaviour.

This is a binary classification problem.

- One row should represent one customer.
- Output feature: `is_student`
- Input features are behavioural summaries derived from baskets, basket items, products and transactions.

Example candidate features:

Feature	Rationale	Possible implementation
Time of day they shop	Students may have fewer constraints on shopping time.	Mean and variance of shopping time.
Amount spent per shop	Students may have lower budgets.	Mean / standard deviation of basket value.
Range of products purchased	Students may be less likely to shop for large families.	Mean / median distinct items per basket.
Habitual behaviour	Could indicate personality or routine.	Count of never-purchased-before items per basket.
Visit regularity	Possible loyalty indicator.	Average gap in days between visits.

The important point is that the feature definitions are tied to a reasoning process, not just created randomly.

5. Always Define What a Row Represents

A repeated theme in the lecture is:

Ask what a row represents before and after every query.

Examples:

- In the `customers` table, one row represents one customer.
- In the `baskets` table, one row represents one basket / receipt.
- In the `basket_items` table, one row represents one line item within a basket.
- After joining `baskets` and `basket_items`, one row represents one basket line item extended with basket information.
- After grouping by customer, one row can represent one customer with aggregated features.

This matters because machine learning usually needs one row per instance. If the instance is a customer, the final feature table must contain one row per customer.

6. Simple Output Feature Construction

For the output feature `is_student`, the required data is already in the `customers` table.

The SQL is simple:

```
SELECT customer_id, is_student
FROM customers;
```

This works because:

- the data is in one table,
 - each row already represents a customer,
 - the feature is already directly represented,
 - no aggregation or join is needed.
-

SQL Concepts for Feature Construction

7. Aggregate Functions and GROUP BY

Aggregate functions summarise multiple rows into one result.

Common aggregate functions include:

- COUNT
- SUM
- AVG
- MIN
- MAX
- STD
- VAR
- MEDIAN

GROUP BY creates buckets of rows, and aggregate functions operate within each bucket.

Example idea:

```
SELECT customer_id, COUNT(*) AS visit_count
FROM baskets
GROUP BY customer_id;
```

This turns many basket rows per customer into one customer-level feature.

The lecture's aggregate example explains that several aggregate functions can be applied to each bucket at once.

8. Data Types and Casting

A table column has a data type, such as:

- integer,
- numeric,
- text,
- date,

- timestamp,
- time,
- spatial types.

Data types matter because functions and operations depend on them.

A `CAST` asks the computer to convert a value from one type to another. Not all casts are meaningful. For example, converting `'abc'` to an integer may return `NULL` in SparkSQL but may cause an error in other systems.

Example:

```
CAST(purchased_at AS DATE)
```

or in PostgreSQL style:

```
purchased_at::TIME
```

Important exam point:

Do not assume all SQL dialects handle data types the same way. SparkSQL, PostgreSQL and other systems may differ.

9. Data Manipulation in SELECT

SQL can transform data directly in the `SELECT` clause using:

- functions,
- operators,
- string concatenation,
- date/time extraction,
- casts,
- arithmetic expressions.

The lecture shows that values can be transformed at output time. For feature construction, this is how raw values become useful model inputs.

Example:

```
SELECT customer_id,  
       quantity * value AS line_value  
FROM transaction_data;
```

10. TIMESTAMP Challenges

Timestamp data is often tricky because it contains both date and time.

For the feature “time of day they shop,” we want time but not date.

In PostgreSQL, this may be easy because it supports a `TIME` type:

```
SELECT customer_id,  
       AVG(purchased_at::TIME),  
       STD(purchased_at::TIME)  
FROM baskets  
GROUP BY customer_id;
```

In SparkSQL, the lecture notes that there is no separate `TIME` datatype in the same way. The workaround is to create a fixed-date timestamp using the time part, convert it to a Unix timestamp, and then calculate averages or standard deviations.

Example idea:

```
SELECT customer_id,  
       AVG(unix_timestamp(date_format(purchased_at, '2022-01-01 hh:mm:ss')) AS  
tod_avg,  
       STD(unix_timestamp(date_format(purchased_at, '2022-01-01 hh:mm:ss')) AS  
tod_std  
FROM baskets  
GROUP BY customer_id;
```

The result is still in Unix timestamp representation, but it can be used as a numeric input feature.

Joins and Feature Construction

11. What a JOIN Means

A join creates a new table by combining rows from two tables.

The lecture stresses that after a join, each output row should represent a meaningful new entity whose facts are jointly true.

Before joining, ask:

- What does each table represent?
- What will one joined row represent?
- Are the join keys trustworthy?
- Is the relationship one-to-one, one-to-many, or many-to-many?
- Could the join duplicate rows?
- Could the join lose rows?

12. One-to-One Join

In a one-to-one relationship, each row in one table matches at most one row in the other table.

Example:

- student table,
- contact table.

The result can be interpreted either as:

- student records extended with contact information,
- or contact records extended with student information.

One-to-one joins are usually straightforward.

13. One-to-Many Join

In a one-to-many relationship, one row from the "one" table can match many rows from the "many" table.

Example:

- one customer has many transactions,
- one basket has many basket items.

The result is usually interpreted from the "many" side:

many-side rows extended with one-side information.

Example:

Joining customers to transactions gives transaction rows extended with customer information.

This is crucial because if your final machine learning row should be one customer, you usually need aggregation after joining.

14. Many-to-Many Relationships

The lecture notes that many-to-many relationships should not directly exist in a well-designed relational database.

Instead, they are usually represented using a linking table.

Example:

- baskets,
- items,
- line_items linking baskets to items.

A direct many-to-many join can easily create row explosion or ambiguous row meanings. For feature construction, always understand the actual relational structure before joining.

15. Subqueries

A subquery is a query used inside another query.

The lecture explains that the `FROM` section usually contains a table name, but SQL does not care whether that table is a stored table or the result of another query. Since queries return tables, a query can be placed inside `FROM`.

Example pattern:

```
SELECT customer_id,  
       AVG(p_ct) AS range_avg,  
       MEDIAN(p_ct) AS range_med  
FROM (  
  SELECT customer_id,  
         basket_id,  
         COUNT(DISTINCT product_code) AS p_ct  
  FROM baskets  
  JOIN basket_items  
  USING (basket_id)  
  GROUP BY customer_id, basket_id  
) x  
GROUP BY customer_id;
```

This pattern is common when you need two levels of aggregation:

1. basket-level feature,
 2. customer-level summary of that basket-level feature.
-

Building Individual Features

16. Feature: Range of Products Purchased

Goal:

Mean and median of the number of distinct products bought per basket.

Why?

Students may be less likely to buy for large families, so basket variety may help predict student status.

Required tables:

- `baskets`
- `basket_items`

Steps:

1. Join baskets to basket items.
2. Count distinct product codes per customer and basket.
3. Aggregate those basket-level counts to the customer level.

SQL structure:

```
SELECT customer_id,  
       AVG(p_ct) AS range_avg,  
       MEDIAN(p_ct) AS range_med  
FROM (  
  SELECT customer_id,  
         basket_id,  
         COUNT(DISTINCT product_code) AS p_ct  
  FROM baskets  
  JOIN basket_items  
  USING (basket_id)  
  GROUP BY customer_id, basket_id  
) x  
GROUP BY customer_id;
```

This is a classic example of changing the row meaning:

- inner query: one row per customer-basket,
- outer query: one row per customer.

17. Feature: Time of Day They Shop

Goal:

Mean and variance / standard deviation of shopping time of day.

Why?

Students may shop at less regular or less conventional times.

Required table:

- baskets

Steps:

1. Extract time-of-day from purchased_at .
2. Convert it into a numeric representation.
3. Group by customer.
4. Calculate average and spread.

SparkSQL-style idea:

```
SELECT customer_id,  
       AVG(unix_timestamp(date_format(purchased_at, '2022-01-01 hh:mm:ss'))) AS  
tod_avg,  
       STD(unix_timestamp(date_format(purchased_at, '2022-01-01 hh:mm:ss'))) AS  
tod_std  
FROM baskets  
GROUP BY customer_id;
```

18. Feature: How Regularly They Visit

Goal:

Average gap between visits.

Why?

Visit regularity may indicate loyalty, routine or lifestyle.

This is where SQL window functions become useful.

SQL Window Functions

19. What Is a Window Function?

A window function creates a new column by looking at a set of rows related to the current row.

The lecture explains that window functions operate on the result set. You can imagine walking through the result one row at a time with a “window” that shows relevant rows around or within the same partition.

Window functions differ from normal aggregation because they do not necessarily collapse rows. They can add information while keeping row-level structure.

20. OVER Clause

Window functions use `OVER (...)`.

Common components:

```
OVER (  
  PARTITION BY ...  
  ORDER BY ...  
  ROWS BETWEEN ...  
)
```

PARTITION BY

Defines which rows belong in the same window group.

Example:

```
PARTITION BY customer_id
```

This creates a separate window for each customer.

The lecture compares this to grouping into buckets, but unlike normal `GROUP BY`, the rows are still available for row-level calculations.

ORDER BY

Defines the order inside each partition.

Example:

```
ORDER BY purchased_at
```

This is essential when using previous/next row functions such as `LAG` and `LEAD`.

Important: the `ORDER BY` inside the window only orders rows for the window calculation. It does not replace the final query-level `ORDER BY`.

ROWS BETWEEN

Defines the exact frame of rows used around the current row.

Example ideas:

- current row only,
- previous row to current row,
- unbounded preceding to current row,
- previous 3 rows to current row.

This is useful for rolling or cumulative calculations.

21. LAG and LEAD

`LAG` looks at a previous row within the ordered partition.

`LEAD` looks at a future row within the ordered partition.

Example:

```
LAG(purchased_at) OVER (  
    PARTITION BY customer_id
```

```
ORDER BY purchased_at  
)
```

This gives the previous purchase time for the same customer.

The lecture notes that `LAG` returns `NULL` for the first record in a partition because there is no previous record. Similarly, `LEAD` returns `NULL` for the final record.

22. Computing Time Between Visits with LAG

To calculate average gap between visits:

1. Group purchase records by customer and purchase time.
2. Use `LAG(purchased_at)` to get the previous visit.
3. Subtract previous visit time from current visit time.
4. Remove `NULL` gaps.
5. Average the gaps per customer.

SQL pattern from the lecture:

```
SELECT customer_id, AVG(gap)
FROM (
    SELECT customer_id,
           purchased_at,
           purchased_at - LAG(purchased_at) OVER (
               PARTITION BY customer_id
               ORDER BY purchased_at
           ) AS gap
    FROM basket
    JOIN basket_items
    USING (basket_id)
    GROUP BY customer_id, purchased_at
) a
WHERE gap IS NOT NULL
GROUP BY customer_id;
```

The logic is that the previous smaller timestamp is subtracted from the current larger timestamp.

Combining Features into a Machine Learning Table

23. Why Simple INNER JOIN Can Be Wrong

After creating separate feature tables, it is tempting to join them all using normal joins.

Example:

```
SELECT customer_id, is_student, tod_avg, tod_std, range_avg, range_med
FROM output
JOIN f1 USING (customer_id)
JOIN f2 USING (customer_id);
```

The lecture says this is wrong if some customers are missing from one feature table. A normal inner join keeps only rows that match in all tables, so customers with missing feature values may disappear.

This changes the training data and can bias the model.

24. LEFT JOIN for Feature Assembly

When the output table defines the set of machine learning instances, use it as the base table and `LEFT JOIN` features onto it.

This keeps all target rows.

Pattern:

```
SELECT ...
FROM output
LEFT JOIN f1 USING (customer_id)
LEFT JOIN f2 USING (customer_id)
LEFT JOIN f3 USING (customer_id);
```

This is important when the target table should define the population of examples.

25. COALESCE for Missing Feature Values

`COALESCE(a, b)` returns the first non-null value.

For feature construction, it is often used to replace missing feature values after a `LEFT JOIN`.

Example:

```
COALESCE(tod_avg, 0) AS tod_avg
```

Meaning:

- if `tod_avg` exists, use it,
- otherwise use `0`.

The lecture's final feature-combination example uses `LEFT JOIN` and `COALESCE` to preserve customers and fill missing feature values.

Exam warning:

Only replace missing values with 0 when 0 is meaningful. Otherwise, choose a better missing-value strategy.

Practical — Feature Construction Exercises

26. Practical Setup

The practical uses a Databricks SQL environment and a retail-style database. It loads tables such as:

- campaign_desc
- campaign_table
- causal_data
- coupon
- coupon_redempt
- hh_demographic
- product
- transaction_data

The practical begins by reminding students to understand the database schema and row meanings before writing queries. It notes that in this data, basket and basket-items concepts are combined into `transaction_data`, so each line in that table must be interpreted carefully.

27. Practical Task Style

The practical asks students to construct features for a prediction task.

Example task:

Predict whether a household is a single-adult household or not from non-household data.

This restriction matters: you cannot use direct demographic household features as inputs if they make the problem trivial or leak the answer.

Example feature definitions include:

Feature	Rationale	Implementation idea
Coupon use	Frugality	Count redeemed coupons per household.
Departments purchased from	Product variety	Count distinct departments purchased from.
Items in a basket	Shop size	Mean and standard deviation of basket quantities.

Feature	Rationale	Implementation idea
Range of products	Product range	Mean / median distinct items per basket.
Stores visited per day	Mobility	Unique stores divided by active days.
Monetary value of baskets	Income proxy	Mean and standard deviation of basket value.
Visit regularity	Loyalty / routine	Mean and variance of gaps between visits.
Habitual items	Personality / routine	Median count of never-before-bought items per basket.
Habitual brands	Personality / routine	Median count of never-before-bought brands per basket.

The key skill is to convert a feature definition into SQL while preserving the intended row meaning.

Session — Working with Temporal Data I

28. Non-Temporal Problems with Temporal Features

A major conceptual distinction in Week 5:

Using temporal features does not automatically mean the task is a temporal prediction problem.

Example:

- Predict where someone currently lives.
- Inputs may include salary over the last year or cities visited in the last month.
- Output is still a current property, not a future quantity.

The lecture states that these are still non-temporal prediction problems if the output property is fixed at a reference point. Temporal features are built relative to that reference point.

29. Reference Date

Temporal features require a fixed reference point.

A reference date defines "now" for the feature construction process.

Example:

- survey date: 10/02/2019,

- reference date: 2015-01-31,
- end of records date.

All temporal features are calculated relative to that date.

Examples:

- salary in the last year,
- bank balance at the reference date,
- visits in the last 30 days,
- days since last purchase,
- spend in the previous 3 months.

The lecture emphasises that all data must be collected relative to the same reference point.

30. Types of Temporal Features

The lecture identifies three main types of temporal features:

1. **Point features**
 2. **Global aggregate features**
 3. **Window aggregate features**
-

31. Point Temporal Features

A point feature describes the value at a specific time or current period.

Examples:

- current salary,
- current bank balance,
- current employment status,
- current age at reference date,
- days since opened.

Point features are useful but may throw away historical patterns.

32. Global Aggregate Features

A global aggregate summarises all history before the reference date.

Examples:

- mean salary since age 16,
- mean bank balance over all records,
- total spend to date,

- total visits to date.

Global aggregates are simple, but they can hide important changes over time.

For example, a customer who shopped heavily years ago but stopped recently may look similar to someone who shops moderately and consistently.

33. Window Aggregate Features

Window features split an entity's history into time periods and calculate aggregates for each period.

The lecture says windows break an entity's history into multiple periods, and an aggregate over each period becomes one or more input features.

Examples:

- spend in last 30 days,
- spend 30–60 days before reference date,
- visits in each of the last 4 weeks,
- milk sales in each previous 3-day chunk.

Window features preserve temporal patterns better than a single global aggregate.

Window Types

34. Tumbling Windows

Tumbling windows split the time series into non-overlapping chunks.

You must define:

- chunk size,
- number of chunks.

Example:

For 3-day windows before a reference date:

- sales 6–9 days before reference date,
- sales 3–6 days before reference date,
- sales 0–3 days before reference date.

The lecture's milk-sales example shows a global aggregate of 75 units, but tumbling windows split that into 46, 15 and 14 units, revealing temporal pattern that the global total hides.

Strengths

- less correlation than expanding windows,

- captures time-local behaviour,
- useful for recent versus older behaviour.

Weaknesses

- edge effects,
- arbitrary boundaries can change feature values,
- can create many features.

The lecture shows that changing the definition of a "day" from 00:00–23:59 to 09:00–08:59 can change which window a large transaction belongs to, creating different feature values.

35. Expanding Windows

Expanding windows grow backward from the reference date.

Example:

- sales 0–3 days before reference date,
- sales 0–6 days before reference date,
- sales 0–9 days before reference date.

Strengths:

- limited edge effects,
- useful for cumulative history.

Weaknesses:

- high correlation between features,
- lots of features.

The lecture explicitly contrasts tumbling and expanding windows: tumbling has edge effects but less correlation; expanding has limited edge effects but more correlation.

36. Sliding Windows

Sliding windows overlap and move step by step through time.

The lecture mentions sliding windows but says they are generally bad in this context and are not covered in detail.

For exam purposes, know:

- they can capture detailed temporal movement,
- but they create highly correlated features,
- they can generate many features,
- they are more complex to use safely.

37. Why Not Only Use Point or Global Aggregates?

The lecture warns that point features and global aggregates throw away information.

They may hide:

- shopping regularity,
- habits,
- someone stopping shopping,
- time-of-day patterns,
- behavioural change.

The lecture states that temporal patterns often contain significant information that can improve analytics.

Temporal Feature Construction in SQL

38. Creating Window Features with CASE

The temporal lecture gives an example:

Predict milk consumption for a specific past week across FoodCorp stores.

This is not future prediction because the week is in the past. The goal is to fill missing records for stores where staff did not record sales.

The examples:

- use tumbling windows,
- use a univariate time series,
- calculate quantity of milk bought over each period,
- use a reference date,
- use `CASE` statements to place records into the right window.

A simplified SQL pattern:

```
SELECT store_id,  
       SUM(CASE WHEN day >= RD - 9 AND day < RD - 6 THEN quantity ELSE 0 END) AS  
sales_6_9,  
       SUM(CASE WHEN day >= RD - 6 AND day < RD - 3 THEN quantity ELSE 0 END) AS  
sales_3_6,  
       SUM(CASE WHEN day >= RD - 3 AND day < RD      THEN quantity ELSE 0 END) AS  
sales_0_3  
FROM transaction_data  
WHERE product = 'milk'  
GROUP BY store_id;
```

The key idea is:

Each CASE expression creates one time-window feature.

39. Creating the Output Label

The output label is based on the prediction definition.

Usually, for temporal dataset construction, the output may be an aggregate over one future chunk relative to the reference date.

Example:

- input features: sales in previous windows,
- output label: sales in the next window after the reference date.

The lecture says adding the output feature can be done with one extra `CASE` statement.

Important:

For true forecasting, you must make sure the input windows occur before the output window. Otherwise, you create leakage.

40. SQL from Python

The temporal lecture also shows that SQL can be parameterised from Python.

Pattern:

- create SQL as a string,
- use placeholders for parameters such as reference date,
- call `read_sql(sql, conn, params)`.

This allows you to generate temporal features for arbitrary reference dates instead of hard-coding dates.

In Databricks/SparkSQL, syntax differs because queries run through the Spark context rather than a standard database connection.

Week 5 High-Yield Exam Checklist

You should be able to explain:

- why SQL is useful for feature construction,
- why loading all tables into Python can be a bad idea,
- how to define what one row represents,
- how to design features from a prediction task,
- how `GROUP BY` turns many rows into one feature row,

- why data types and casting matter,
 - why timestamps are difficult,
 - how to extract time-of-day features,
 - how joins change row meaning,
 - one-to-one, one-to-many and many-to-many relationships,
 - why subqueries are useful for multi-level aggregation,
 - why inner joins can accidentally remove training examples,
 - why `LEFT JOIN` is used when assembling feature tables,
 - why `COALESCE` is used after left joins,
 - how SQL window functions work,
 - what `PARTITION BY`, `ORDER BY`, `ROWS BETWEEN`, `LAG` and `LEAD` do,
 - how to compute average time between visits,
 - what temporal features are,
 - why temporal features can be used in non-temporal prediction,
 - what a reference date is,
 - point, global aggregate and window aggregate features,
 - tumbling versus expanding windows,
 - edge effects and correlation trade-offs,
 - how `CASE` statements create time-window features,
 - how output labels are created relative to a reference date.
-

Part B — Q&A Summary

1. What is the main purpose of Week 5?

To learn how to construct machine learning input and output tables from relational and temporal data using SQL.

2. Why is SQL useful for machine learning feature construction?

Because business data is often stored in databases, and SQL is designed to transform, join and aggregate tabular data efficiently.

3. What does “tables in, table out” mean?

It means using SQL to process raw database tables and directly produce the final machine learning feature table, rather than moving all raw data into Python first.

4. Why might loading all tables into Python be a bad idea?

It can cause memory problems, unnecessary data transfer, more complex code and inefficient processing.

5. What are the first questions to ask before writing feature SQL?

Ask what you are predicting, what one row should represent, what input features you want, why you want them, what format they need, and where the information comes from.

6. In the student-prediction example, what does one row represent?

One customer.

7. What is the output feature in the student-prediction example?

`is_student` , a binary target indicating whether the customer is a student.

8. Why is "time of day they shop" a useful feature?

Because students may have fewer constraints on when they shop, so shopping time patterns may help prediction.

9. Why is "amount spent per shop" useful?

Because students may have reduced budgets, so spending behaviour may differ.

10. Why is "range of products purchased" useful?

Because students may be less likely to buy products for a large family, so basket variety may differ.

11. Why is "how regular they visit" useful?

Because visit gaps may indicate loyalty, routine or lifestyle.

12. Why must we know what a row represents?

Because machine learning requires one row per instance, and joins/aggregations can change row meaning.

13. What does `GROUP BY` do?

It groups rows into buckets and allows aggregate functions to summarise each bucket.

14. Give examples of aggregate functions.

COUNT , SUM , AVG , MIN , MAX , STD , VAR and MEDIAN .

15. What does COUNT(DISTINCT product_code) do?

It counts the number of unique product codes in a group.

16. What is a subquery?

A query used inside another query, usually as a temporary table in the FROM clause.

17. Why are subqueries useful in feature construction?

They allow multi-level aggregation, such as first creating basket-level features and then summarising them at customer level.

18. Why are data types important in SQL?

Because operations and functions depend on the type of value being processed.

19. What is casting?

Casting converts a value from one data type to another.

20. Why are timestamps difficult?

A timestamp contains both date and time, but sometimes the feature only needs one component, such as time of day.

21. How can PostgreSQL extract time from a timestamp?

By casting to TIME , for example purchased_at::TIME .

22. Why does SparkSQL need a workaround for time-of-day features?

Because it does not support a separate TIME datatype in the same way, so time must be transformed into a numeric representation.

23. What is a SQL join?

A join combines rows from two tables to create a new table.

24. What should you ask before joining tables?

Ask whether the joined row represents a meaningful entity and whether the matched facts are jointly true.

25. What is a one-to-one join?

Each row in one table matches at most one row in the other table.

26. What is a one-to-many join?

One row in one table can match many rows in another table.

27. How should a one-to-many join usually be interpreted?

As rows from the many side extended with information from the one side.

28. Why are many-to-many relationships dangerous?

They can create ambiguous row meanings or row explosion if not handled through a linking table.

29. Why can an inner join be wrong when assembling features?

It drops rows that do not appear in every feature table, possibly removing valid training examples.

30. What is a `LEFT JOIN` used for?

To keep all rows from the base table while adding matching feature values where available.

31. What should usually be the base table when assembling the final ML table?

The table defining the target population and output feature.

32. What does `COALESCE` do?

It returns the first non-null value from a list.

33. Why use `COALESCE` after a `LEFT JOIN` ?

To replace missing feature values with a chosen default, such as 0, when appropriate.

34. Is replacing missing values with 0 always correct?

No. It is only correct when 0 is a meaningful replacement.

35. What is a SQL window function?

A function that calculates a value using a set of rows related to the current row, without necessarily collapsing the result into fewer rows.

36. What does **OVER** do?

It defines the window over which the window function operates.

37. What does **PARTITION BY** do?

It divides rows into groups or buckets for window calculations.

38. What does **ORDER BY** do inside a window function?

It defines the order of rows within each partition for the window calculation.

39. Does window **ORDER BY** control the final output order?

No. It only controls the order used inside the window function.

40. What does **LAG** do?

It returns a value from a previous row in the same ordered partition.

41. What does **LEAD** do?

It returns a value from a following row in the same ordered partition.

42. Why is **LAG** useful for visit regularity?

It lets us compare a current visit time with the previous visit time for the same customer.

43. Why is the first **LAG** value usually null?

Because there is no previous record for the first row in a partition.

44. How do we compute average time between visits?

Use `LAG` to find the previous visit, subtract it from the current visit, remove null gaps, and average the gaps by customer.

45. What is the main practical SQL pattern for “range of products purchased”?

First count distinct products per basket, then average or take the median of those counts per customer.

46. What is a non-temporal prediction problem?

A task where the output is a fixed property of an instance, not a future value.

47. Can non-temporal prediction use temporal features?

Yes. Temporal histories can be used as inputs even when the output is not a future quantity.

48. Give an example of a non-temporal problem with temporal features.

Predicting someone’s current home area using income over the last year or cities visited in the last month.

49. What is a reference date?

A fixed date that defines the point relative to which temporal features are constructed.

50. Why is a reference date important?

It ensures all temporal features are calculated consistently and prevents using information from the wrong time period.

51. What are point temporal features?

Features measured at a specific time or current period.

52. Give examples of point temporal features.

Current salary, current bank balance, current employment status or age at reference date.

53. What are global aggregate temporal features?

Summaries over the full available history before the reference date.

54. Give examples of global aggregates.

Total spend to date, mean bank balance, total visits or mean salary since age 16.

55. What are window aggregate features?

Features created by splitting history into time periods and calculating aggregates for each period.

56. Why are window features useful?

They preserve temporal patterns that point features and global aggregates may hide.

57. What is a tumbling window?

A non-overlapping time chunk.

58. What must be defined for tumbling windows?

The chunk size and number of chunks.

59. Give an example of tumbling windows.

Sales 6–9 days before reference date, 3–6 days before reference date, and 0–3 days before reference date.

60. What is an advantage of tumbling windows?

They create less correlation than expanding windows.

61. What is a disadvantage of tumbling windows?

They suffer from edge effects because arbitrary boundaries can change feature values.

62. What are edge effects?

Cases where small changes in time-window boundaries cause large changes in feature values.

63. What is an expanding window?

A window that grows cumulatively backward from the reference date.

64. Give an example of expanding windows.

Sales 0–3 days before reference date, 0–6 days before reference date, and 0–9 days before reference date.

65. What is an advantage of expanding windows?

They have limited edge effects.

66. What is a disadvantage of expanding windows?

They create highly correlated features.

67. What is a sliding window?

An overlapping window that moves through time step by step.

68. Why are sliding windows risky in this context?

They can create many highly correlated features and are more complex to use safely.

69. Why not only use global aggregates?

Global aggregates can hide temporal behaviour such as recency, habits, stopping shopping or changing routines.

70. What is the milk-sales example used to show?

It shows how temporal sales history can be converted into windowed features using a reference date and tumbling windows.

71. Why is the milk-sales example not necessarily future prediction?

Because the target week may be in the past; the task can be to estimate missing historical sales.

72. How can SQL create window-based temporal features?

Use `CASE` statements inside aggregate functions to sum or count records belonging to each time window.

73. How is an output label added in temporal feature construction?

Usually by adding another aggregate or `CASE` statement for the target period.

74. What is the danger when creating temporal labels?

Using future information inside input features causes data leakage.

75. How can SQL be reused for different reference dates?

Parameterise the SQL from Python and pass the reference date as a parameter.

76. What is the main exam message of Week 5?

Feature construction is not just writing SQL. You must define the prediction task, row meaning, feature rationale, joins, aggregation levels, temporal reference points, and leakage risks before building the final machine learning table.